Learned Page Replacement in xv6

Week 16 update: kernel results against the classical policies

Summary

The question: which learned policy works best inside the xv6 kernel, and does it beat the classical policies (Clock, and now ARC, LHD and S3-FIFO as well)? Everything in the main text was measured in the real xv6 kernel running on QEMU; the simulator results are in Appendix B. Numbers are page faults relative to Clock (below 1 is better), averaged with a geometric mean over the streams of each workload and then over the 14 workloads (so a workload with 17 streams counts once, like one with 2). Memory is 5, 10 and 20% of each program's pages. Intervals are 95% intervals from resampling the workloads, then the streams inside them.

- **Classical policies, in the kernel.** ARC, LHD and S3-FIFO now run in the kernel, in the form a kernel that only sees faults and accessed bits can run (Section 1.1). **ARC beats Clock at 5 and 10% memory** (0.947 and 0.943, intervals below 1) and is level at 20% (0.984). LHD is level at 5% (1.005), and 0.95 at 10 and 20% with intervals that touch 1. S3-FIFO is not better than Clock at any size (1.09, 1.02, 1.09). FIFO, Aging and LFU are worse than Clock at every size (by 4 to 35%).
- **The linear model.** A small linear score over kernel-visible signals, with whole-number arithmetic, beats Clock at 10% memory (0.914, interval 0.85 to 0.98). At 5% (0.976) and 20% (0.882) it is better on average but the intervals include 1 (at 20% only just: the upper end is 1.001): a few workloads decide the average (it is bad on graph and matmul at some sizes, very good on the hash join at 20% memory and on the heap programs).
- **Learned against the best classical policy.** The linear model against ARC: 1.031, 0.970 and 0.896 at 5, 10 and 20% memory; no interval excludes 1. The gap moves in the learned model's favour as memory grows. **We have not shown that a learned policy beats ARC on average**; we have shown that it beats Clock at 10%, and that it is at least level with ARC.
- **Heavier models.** The five kinds trained on all workloads (CNN, deeper MLP, set-attention, boosted trees, ranking trees) were run on all 74 streams at 10% memory (and on ten streams at 5 and 20%). At 10% the CNN, the MLP and set-attention are level with the linear model (1.000, 0.998 and 0.992; every interval contains 1), and the CNN and the MLP are better than Clock (0.914 and 0.912, intervals below 1). Boosted trees are slightly worse than the linear model (1.061, interval 1.01 to 1.12) and ranking trees clearly worse (1.476, 1.05 to 2.42). On ten streams at 20% memory the first three are 12 to 14% better than ARC, but with intervals that include 1. They cost 20 to 150 times the linear model per eviction (Section 4), so we found no reason to prefer them.
- **Looking at fewer candidates costs little.** The linear score scanning only 32 randomly chosen candidates per eviction is level with scanning all of them (0.998 of the full score, interval 0.975 to 1.017) and still beats Clock (0.912, interval 0.86 to 0.97) at about half the cost (11 300 instead of 22 800 instructions at 68 candidates). With 16 candidates it is 1.3% worse than the full score (1.013, 0.98 to 1.04) and still beats Clock (0.926, 0.88 to 0.98) at a quarter of the cost (5 700 instructions). All 74 streams, 10% memory. Appendix C compares other ways of choosing the pages.
- **Training on all workloads is the fair test.** Trained on one workload, the five kinds are mostly worse than when trained on all (set-attention 1.48 against 0.90 in the simulator); models trained on one workload answer a different question and are kept only as a side result.
- **The kernel agrees with the simulator.** For the three new policies, kernel faults divided by simulator faults are 0.995 (ARC), 1.014 (S3-FIFO) and 0.984 (LHD), geometric mean over 74 streams at 10% memory; the median stream is within 1.5%.

1 What was run

The kernel and the metric. Each run is one program in xv6 with its resident memory limited to a fraction of the program's pages, so every eviction goes through the policy under test. The count is zero-fill faults plus swap faults. xv6 runs are exactly repeatable (the same program, stream, memory size and policy give the same faults every time); the uncertainty in the tables comes from which streams were chosen, not from run-to-run noise. 2 743 runs are merged in one file (`report/results2/kernel_eval_all.csv`); the long runs were split between the thesis PC and Kaggle notebooks, and 180 runs repeated on both gave identical faults, evictions and writes.

Workloads (14). B-tree, graph, key-value store (the original and the Twitter replays, "kv-T"), LZW, matrix multiply, sort, heap churn, hash join, sparse matrix–vector product (SpMV), pointer chase, Bloom filter, the synthetic pattern suite (`pat`) and four application-shaped programs (`app`). 74 streams at each of the three memory sizes. Every program's access string is checked reference by reference against an independent model.

Policies in the kernel. FIFO, Clock, Aging, LFU; the learned linear score (`global`: one set of weights for all workloads; a per-workload set exists too); five heavier models trained on all workloads (the six original ones), converted to whole-number arithmetic; and ARC, LHD and S3-FIFO.

1.1 The three classical policies in the kernel

A kernel does not see every access. It sees faults, and the accessed and dirty bits of a page when it looks. ARC, LHD and S3-FIFO as published react to every hit, so they cannot be run unchanged. Each was adapted in the same way: the accessed bits are read, and cleared, at each eviction, and a page found accessed counts as having been hit since the last look (the touch that loaded a page is ignored). Everything else follows the original: for ARC the four lists, the ghost lists and the adaptation of the target size; for S3-FIFO the small queue (a tenth of memory), the main queue and the ghost queue; for LHD the hit and eviction histograms per class and age, with time counted in evictions. The simulator versions were checked against independent reference implementations (570 traces, with identical faults and evictions on all). They are named "ARC" in the tables, but they are **not** exactly ARC: the S3-FIFO result in particular (not better than Clock) should be read as a result for this adapted form, not for the original algorithm.

2 Main result: every policy, three memory sizes

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.12	1.06	1.42	1.45	1.18	2.01	1.36	1.07	1.07	1.09	1.12	1.11	1.14	1.27	1.23
Aging	1.07	1.02	1.06	1.07	1.02	1.31	0.98	1.01	1.01	1.00	1.06	1.00	1.04	0.99	1.04
LFU	1.09	1.03	1.16	1.18	1.04	1.31	0.99	1.01	1.01	1.04	1.06	1.00	1.05	1.02	1.07
S3-FIFO	0.96	0.99	1.05	1.11	1.07	1.61	1.50	0.86	1.05	1.09	0.97	1.01	0.94	1.23	1.09
LHD	0.96	0.99	0.96	0.99	0.94	1.18	1.10	1.00	1.00	1.00	0.97	1.00	0.98	1.01	1.00
ARC	0.89	1.02	0.86	0.88	0.97	0.99	0.97	0.91	0.99	1.00	0.91	1.00	0.87	1.01	0.95
linear, global	0.84	1.01	0.82	0.83	0.95	1.72	1.03	0.86	0.98	1.10	0.88	1.00	0.87	1.04	0.98

Table 1: 5% memory. Faults relative to Clock in xv6; one column per workload (geometric mean over its streams); last column: mean over the 14 workloads.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.11	1.64	1.29	1.31	1.13	1.82	1.24	1.04	1.04	1.63	1.09	1.06	1.19	1.22	1.25
Aging	1.09	1.42	1.12	1.12	1.05	1.00	0.99	1.02	1.01	1.00	1.06	1.00	1.14	0.98	1.07
LFU	1.10	1.61	1.17	1.18	1.06	1.00	1.00	1.02	1.01	1.58	1.06	1.00	1.14	1.09	1.13
S3-FIFO	0.99	1.11	1.08	1.11	1.11	0.90	1.05	0.76	1.08	1.05	0.99	1.01	0.97	1.08	1.02
LHD	0.97	0.63	1.00	1.04	0.87	0.87	1.04	1.01	0.99	1.03	0.98	1.00	1.00	0.99	0.95
ARC	0.89	0.86	0.89	0.90	0.98	0.97	1.01	0.85	1.00	1.09	0.92	1.00	0.88	0.99	0.94
linear, global	0.84	0.74	0.88	0.85	0.94	1.10	0.97	0.73	0.99	1.12	0.86	1.00	0.87	1.00	0.91

Table 2: 10% memory, same layout (74 streams).

- **ARC** is best among the classical policies at small memory (0.85–0.91 on btree, kv, heap and the Twitter replays at 5 and 10%); on graph it gives 0.86 at 10% and 1.02 and 1.03 at 5 and 20%, while LHD does very well at 10% (0.63), and on matmul and sort the learned model is worse than Clock at 5% (1.72 and 1.03) while ARC is level.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
FIFO	1.11	1.74	1.23	1.25	1.60	1.58	1.14	0.90	2.38	2.11	1.08	1.03	1.12	1.31	1.35
Aging	1.10	1.56	1.14	1.15	1.45	1.11	1.00	0.88	2.29	1.03	1.06	1.00	1.10	0.98	1.17
LFU	1.10	1.71	1.17	1.18	1.55	1.11	1.00	0.93	2.35	2.09	1.07	1.00	1.10	1.19	1.27
S3-FIFO	1.05	1.38	1.14	1.15	1.61	1.11	0.99	0.68	1.08	1.04	1.02	1.01	1.00	1.19	1.09
LHD	0.96	0.99	1.00	1.05	0.85	0.59	1.02	1.08	0.90	1.00	0.97	1.00	1.00	0.98	0.95
ARC	0.93	1.03	0.93	0.93	0.94	1.08	1.03	1.04	1.01	1.04	0.94	1.00	0.92	0.98	0.98
linear, global	0.90	1.40	0.96	0.89	0.95	0.92	0.90	0.56	0.55	0.83	0.85	1.00	0.88	1.09	0.88

Table 3: 20% memory, same layout.

- **The linear model** does best where recency is a poor guide: btree and kv (0.82–0.96 at all sizes), heap and chase, and the hash join at 20% (0.55). Its failures are specific and repeat across sizes: matmul at 5% (1.72; only 2 frames), SpMV at 5 and 10% (1.10, 1.12), graph at 5 and 20% with the global weights (1.01 and 1.40; the graph-specific weights score 0.60 at 10%).
- **LHD** wins clearly on one workload, graph at 10% (0.63), and on LZW and matmul at 20%; it is the most uneven.
- **LeCaR, a learned-cache policy from the literature, in the kernel** (the form a kernel can run, 74 streams at 10% memory): **worse than Clock**, 1.113 of its faults (interval 1.04 to 1.23) and 1.20 of its page writes; the linear score is 0.822 of LeCaR's faults (0.73 to 0.91). It matches the simulator (kernel faults divided by simulator faults: 1.00 geometric mean, median 1.004 over the 59 streams with the same length). It is very bad on graph (1.60 of Clock) and SpMV (1.47). Choosing a victim costs 8 200 instructions, 0.36 times the linear score.
- **No policy helps** on Bloom filter (all 1.00) or the application programs at 5%.

comparison	5% memory	10% memory	20% memory
ARC against Clock	**0.947 [0.92, 0.98]**	**0.943 [0.90, 0.98]**	0.984 [0.95, 1.02]
LHD against Clock	1.005 [0.98, 1.05]	0.951 [0.88, 1.00]	0.949 [0.87, 1.01]
S3-FIFO against Clock	1.086 [1.00, 1.22]	1.016 [0.96, 1.07]	1.086 [0.98, 1.20]
linear, global against Clock	0.976 [0.90, 1.09]	**0.914 [0.85, 0.98]**	0.882 [0.77, 1.00]
linear, global against ARC	1.031 [0.97, 1.13]	0.970 [0.92, 1.02]	0.896 [0.77, 1.02]
linear, global against LHD	0.972 [0.91, 1.06]	0.961 [0.89, 1.04]	0.929 [0.80, 1.08]
LeCaR against Clock	–	1.113 [1.04, 1.23]	–
linear, global against LeCaR	–	**0.822 [0.73, 0.91]**	–
linear, 32 sampled against Clock	–	**0.912 [0.86, 0.97]**	–
linear, 16 sampled against Clock	–	**0.926 [0.88, 0.97]**	–
linear, 32 sampled against linear, global	–	0.998 [0.97, 1.02]	–
linear, 16 sampled against linear, global	–	1.013 [0.98, 1.04]	–
CNN against Clock	0.932 [0.86, 1.00]	**0.914 [0.84, 0.97]**	**0.829 [0.68, 0.94]**
deeper MLP against Clock	0.954 [0.87, 1.06]	**0.912 [0.84, 0.97]**	**0.838 [0.70, 0.94]**
CNN against ARC	0.999 [0.96, 1.04]	0.969 [0.90, 1.02]	0.864 [0.69, 1.02]
deeper MLP against ARC	1.024 [0.96, 1.10]	0.968 [0.90, 1.02]	0.873 [0.71, 1.01]
set-attention against ARC	**0.965 [0.94, 0.98]**	0.962 [0.89, 1.02]	0.879 [0.73, 1.03]
boosted trees against ARC	0.997 [0.97, 1.02]	1.029 [0.97, 1.09]	0.901 [0.72, 1.05]
CNN against linear, global	0.981 [0.90, 1.05]	1.000 [0.94, 1.06]	1.028 [0.94, 1.13]
deeper MLP against linear, global	1.006 [0.97, 1.04]	0.998 [0.94, 1.06]	1.039 [0.96, 1.12]
set-attention against linear, global	0.948 [0.87, 1.01]	0.992 [0.93, 1.04]	1.046 [0.95, 1.14]
boosted trees against linear, global	0.979 [0.89, 1.05]	1.061 [1.01, 1.12]	1.073 [0.99, 1.17]
ranking trees against linear, global	1.149 [0.96, 1.44]	1.476 [1.05, 2.43]	1.092 [0.99, 1.21]

Table 4: The comparisons that matter, with 95% intervals (resampling workloads, then streams). Bold: the whole interval is below 1. At 10% memory every row uses all 74 streams (LeCaR was run at 10% only). At 5 and 20% the rows for the heavier models (CNN, MLP, set-attention, trees) use only the ten streams they were run on there, and the sampled linear scores were run at 10% only.

Reading the intervals. The intervals are wide because the workloads differ a lot, and only 14 workloads are resampled. Bold entries are the ones we would write down as "better": ARC against Clock at 5 and 10%, the linear model against Clock at 10%, the 32- and 16-candidate linear scores against Clock at 10%, the CNN and MLP against Clock at 10% (and at 20% on ten streams), and set-attention against ARC at 5% (ten streams; it is not better than ARC at 10%, so we do not rely on it). Two comparisons go the other way: boosted trees and ranking trees are worse than the linear model at 10% (the intervals lie above 1). Everything else is "level, or too uncertain".

3 The models trained on all workloads

Five kinds of heavier model were trained on all workloads and put in the kernel: a CNN over the aging bits, a deeper MLP, set-attention, boosted trees and ranking boosted trees. They were run on all 74 streams at 10% memory, and on ten streams (one per variant of ten workloads) at 5 and 20%.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.82	–	0.81	0.78	–	1.48	0.98	0.92	0.91	1.14	0.83	1.00	–	–	0.95
ARC	0.89	–	0.83	0.84	–	0.99	1.00	0.94	0.98	1.00	0.88	1.00	–	–	0.93
LHD	0.97	–	0.95	0.98	–	1.40	1.06	0.99	1.01	0.99	0.96	1.00	–	–	1.02
CNN	0.81	–	0.82	0.75	–	1.01	1.08	0.99	0.94	1.09	0.88	1.00	–	–	0.93
deeper MLP	0.81	–	0.80	0.76	–	1.29	1.01	0.99	0.95	1.19	0.88	1.00	–	–	0.95
set-attention	0.84	–	0.79	0.76	–	0.99	0.97	0.91	0.97	1.00	0.82	1.00	–	–	0.90
boosted trees	0.88	–	0.80	0.76	–	1.00	0.97	1.00	0.99	1.05	0.89	1.00	–	–	0.93
ranking trees	1.10	–	0.91	0.78	–	1.03	0.95	0.92	1.03	3.10	1.06	1.00	–	–	1.09

Table 5: 5% memory, the ten streams the heavier models were run on there. Columns as in the main tables; some workloads are missing because they have no stream here.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.84	0.74	0.88	0.85	0.94	1.10	0.97	0.73	0.99	1.12	0.86	1.00	0.87	1.00	0.91
ARC	0.89	0.86	0.89	0.90	0.98	0.97	1.01	0.85	1.00	1.09	0.92	1.00	0.88	0.99	0.94
LHD	0.97	0.63	1.00	1.04	0.87	0.87	1.04	1.01	0.99	1.03	0.98	1.00	1.00	0.99	0.95
CNN	0.83	0.59	0.93	0.85	0.93	0.94	1.07	0.96	0.97	1.01	0.92	1.00	0.89	1.02	0.91
deeper MLP	0.82	0.59	0.93	0.86	0.91	0.94	0.98	0.96	0.99	1.05	0.92	1.00	0.87	1.06	0.91
set-attention	0.87	0.55	0.92	0.93	0.92	0.94	1.02	0.72	0.98	1.18	0.86	1.00	0.94	1.07	0.91
boosted trees	0.92	0.77	0.94	0.86	0.95	1.02	0.97	0.97	1.10	1.28	0.93	1.00	0.99	0.95	0.97
ranking trees	1.10	10.52	1.07	0.88	1.30	0.97	0.94	0.73	0.98	5.81	1.04	1.00	1.19	1.00	1.35

Table 6: 10% memory, all 74 streams, same layout. The mean is over the 14 workloads.

policy	btree	graph	kv	kv-T	lzw	matmul	sort	heap	join	spmv	chase	bloom	pat	app	mean
linear, global	0.91	–	0.94	0.83	–	0.98	0.90	0.69	0.41	0.81	0.80	1.00	–	–	0.81
ARC	0.93	–	0.92	0.88	–	1.12	1.02	0.84	1.04	0.97	0.92	1.00	–	–	0.96
LHD	0.96	–	1.00	1.06	–	0.63	1.01	1.02	0.87	1.01	0.96	1.00	–	–	0.94
CNN	0.91	–	1.05	0.83	–	0.72	0.89	0.96	0.39	0.89	0.90	1.00	–	–	0.83
deeper MLP	0.90	–	1.02	0.83	–	0.80	0.89	0.94	0.40	0.93	0.90	1.00	–	–	0.84
set-attention	0.97	–	1.03	1.01	–	0.72	0.88	0.68	0.46	1.10	0.84	1.00	–	–	0.84
boosted trees	1.00	–	1.03	0.84	–	0.82	0.91	0.97	0.39	1.04	0.91	1.00	–	–	0.86
ranking trees	1.26	–	1.13	0.88	–	0.86	0.85	0.69	0.38	1.06	1.09	1.00	–	–	0.88

Table 7: 20% memory, same layout as 5% (ten streams).

- At 5% the CNN, boosted trees and set-attention are within a few points of ARC (0.93, 0.93, 0.90 against 0.93); at 10% (74 streams) the CNN, the MLP and set-attention match the global linear model (ratios 1.000, 0.998 and 0.992, intervals 0.93 to 1.06); at 20% every kind except ranking trees is about 10 to 14% better than ARC on these streams, but the intervals include 1.
- **Ranking trees are the one kind that loses** (1.09 against Clock at 5% on ten streams; 1.35 at 10% on all 74, with graph at 10.5 times Clock's faults and SpMV at 5.8 times).
- Set-attention is the least stable: 0.90, 0.91 and 0.84 against the others' steadier numbers; earlier work found its result depends strongly on what is in memory at the start.
- **Trained on one workload instead** (the five were first built that way) the picture is worse for four of the five kinds (simulator, 24 cells, faults against Clock):

model	trained on one workload	trained on all workloads
linear, global	–	0.894
CNN on aging bits	0.939	0.878
deeper MLP	0.926	0.886
boosted trees	0.981	0.963
set-attention	1.478	0.895
ranking trees	1.109	1.186

policy	instructions to choose one victim	times the linear model
FIFO	385	0.02
Clock	411	0.02
Aging	8 125	0.36
LFU	8 005	0.35
S3-FIFO	1 395	0.06
ARC	11 934	0.52
LHD	14 199	0.62
LeCaR	8 155	0.36
linear, global	22 803	1.00
boosted trees	464 325	20
ranking trees	449 616	20
set-attention	1 810 012	79
CNN	1 987 567	87
deeper MLP	3 422 321	150

Table 8: Instructions the kernel spends choosing one victim, with 68 candidates (a 64-frame limit), counted exactly in QEMU's instruction-counting mode (repeatable to under 0.3%), and the same as a multiple of the linear model.

4 What choosing a victim costs

Counts are exact, not timings. The linear score costs about 22 800 instructions per eviction at 68 candidates (about 340 per candidate), 55 times Clock. ARC and LHD cost about half to two thirds of that (11 900 and 14 200), S3-FIFO 1 400. LeCaR costs 8 200 (0.36 times the linear score). The heavier models cost 20 times (boosted trees, ranking trees), 79 (set-attention), 87 (CNN) and 150 times (deeper MLP) the linear score. So ARC is the cheaper of the two best policies, and its advantage over the linear score at 20% memory is the thing to watch; the heavier models cost far more than they could save unless a page fault is very expensive. Whether a saved fault pays for the extra work depends on two things we have now measured on the thesis PC. **Device latency** (4 KB random reads, one request at a time, bypassing the file cache): a Samsung 980 NVMe SSD, median 63 µs (99th percentile 134 µs), and a WD Blue 1 TB hard disk, median 8.9 ms (99th percentile 24.5 ms). The hard-disk figure is for reads spread over a 1 GB file; a compact swap area (xv6's is 32 MB) would see shorter seeks and a lower figure, so 8.9 ms is a pessimistic value for a hard disk (a 32 MB test fits in the drive's own cache and measures the cache, not the disk, so it cannot be used). **Speed of the scoring code**: the kernel's own scoring headers compiled for this CPU (one core of a Ryzen 5 5600) take 1.35 µs for 68 candidates with the linear score (20 ns per candidate), 33 µs with boosted trees, 106 µs with set-attention and the CNN, and 206 µs with the deeper MLP. Divided by the kernel's instruction counts that is about 0.06 ns per instruction, 17 times faster than the 1 ns we had assumed; adding the kernel's page-table walk that reads and clears the accessed bits (the same three-level walk and the same counters), the linear score takes 1.52 µs with warm caches and 2.11 µs when the program has flushed the caches between two evictions (0.093 ns per instruction); the heavier models change by under 10%. Using the cold-cache figure for every policy, the linear score is still faster than Clock on the SSD (0.96 of Clock's time, on 9 of 14 workloads) and the CNN, MLP and boosted trees are still much slower (3.9, 5.8 and 1.8). The timing still leaves out the TLB flush after the scan and the locks, so it remains a lower bound. Time spent on faults and on choosing victims, relative to Clock (geometric mean over workloads; below 1 is faster):

policy	faults / Clock	SSD, 1 ns	disk, 1 ns	SSD, 0.059 ns	disk, 0.059 ns
linear score, all candidates	0.914	1.34	**0.92**	**0.94**	**0.91**
linear score, 32 sampled	0.912	1.05	**0.91**	**0.92**	**0.91**
linear score, 16 sampled	0.926	1.00	**0.93**	**0.93**	**0.93**
ARC	0.943	1.17	**0.94**	**0.96**	**0.94**
LHD	0.951	1.23	**0.95**	**0.97**	**0.95**
LeCaR	1.113	1.30	1.11	1.12	1.11
S3-FIFO	1.016	1.04	1.02	1.02	1.02
CNN	0.914	29.94	1.18	2.88	**0.93**
deeper MLP	0.912	50.55	1.36	4.18	**0.94**
set-attention	0.907	27.30	1.15	2.69	**0.92**
boosted trees	0.970	8.47	1.04	1.51	**0.97**
ranking trees	1.350	11.53	1.44	2.08	1.36

SSD: 63 µs per read, hard disk: 8.9 ms (both measured). "1 ns" is a slow core; "0.059 ns" is this PC's core (measured). Bold: faster than Clock. Geometric mean over the 14 workloads; the table is generated from the 10% kernel runs. So the linear score pays on a hard disk with either core, and on the SSD only with a fast core (faster than Clock on 9 of 14 workloads). The CNN, MLP and set-attention pay on neither an SSD nor a slow core, and win only a few percent on a hard disk with a fast core. This counts only device time per fault and choosing a victim; the rest of the fault path and the programs' own work are left out, and the device was read through a test file, not through xv6. Choosing among only 32 sampled candidates makes the linear cost independent of memory size (11 700 instructions) for 0 to 1 point of faults.

5 Does the simulator predict the kernel?

For the three new policies at 10% memory, over 74 streams, kernel faults divided by simulator faults:

policy	geometric mean	median stream	middle 80% of streams
ARC	0.995	1.005	0.98 to 1.03
S3-FIFO	1.014	1.015	0.99 to 1.09
LHD	0.984	1.000	0.94 to 1.03

The simulator here starts from filled memory, as the kernel's benchmarks do. (Started from empty memory, as the first versions did, some models are misjudged: set-attention by up to 14 times.) The kernel's own pages (code and stack, 3 to 9 frames) and capacity cliffs explain most of the remaining differences. Earlier work found the same agreement for FIFO, Aging, LFU and the linear model (within about 4%).

6 Limits and what is not claimed

- **One operating system.** xv6 on QEMU, a teaching kernel with a single-threaded benchmark process at a time. Nothing here says what happens in Linux.
- **Time is measured only in parts.** Instruction counts are exact. Device read latency was measured on one SSD and one hard disk (a device read, not a full xv6 fault, and for the hard disk over a 1 GB file), and the speed of the scoring code on one desktop core, including the page-table walk but not the TLB flush or the locks, so it is a lower bound. How long a real xv6 fault takes on real hardware was not measured.
- **Fourteen workloads.** Some are close variants of each other, and the five heavier models were run on only ten streams at 5 and 20% memory (all 74 at 10%). The intervals show the resulting uncertainty; most differences between the best policies are inside it.
- **Adapted forms.** ARC, LHD, S3-FIFO and LeCaR ran in the form a kernel can run (accessed bits read at evictions), not as published; S3-FIFO's result in particular should not be quoted as the algorithm's.
- **"All workloads" means the six original ones** (btree, graph, kv, LZW, matmul, sort), for the global linear model and the five heavier kinds alike; the heap, hash join, SpMV, pointer chase, Bloom filter, pattern and application programs were not in training. At 5 and 20% memory the heavier models were run on only ten streams, so their intervals there are the widest.
- **The learned policy can be worse than Clock** (matmul at 5%, SpMV, graph with the global weights at 20%); the tables show where.

A Break-even by workload

The fault cost above which a policy beats Clock on one workload, at 10% memory, on this PC's core (0.059 ns per instruction): the extra instructions spent choosing a victim, divided by the faults saved per eviction. Compare with the measured read times of 63 µs (SSD) and 8.9 ms (hard disk). "never" means the policy does not save faults on that workload.

workload	linear score faults	linear score break-even μs	ARC faults	ARC break-even μs	CNN faults	CNN break-even μs
app	1.004	60.3	0.991	69.5	1.016	7 187
bloom	0.999	1 256	1.0	never	1.0	555 132
btree	0.841	33.7	0.894	26.1	0.831	2 583
chase	0.864	11.5	0.922	9.6	0.921	1 578
graph	0.736	8.2	0.858	3.4	0.59	586
heap	0.735	5.1	0.853	4.2	0.955	3 350
join	0.987	32.1	0.996	55.7	0.972	14 664
kv	0.879	9.5	0.892	5.0	0.932	1 829
kv-T	0.846	6.6	0.9	4.6	0.851	612
lzw	0.941	15.9	0.982	28.8	0.93	1 174
matmul	1.104	never	0.968	2.3	0.936	292
pat	0.873	14.8	0.877	8.1	0.895	1 158
sort	0.967	6.2	1.014	never	1.072	892
spmv	1.124	never	1.086	never	1.007	never

Table 9: Faults against Clock and break-even fault cost, per workload (kernel runs at 10% memory).

B Simulator results

The simulator replays the same streams much faster and was used to choose models. These are the results behind the choices; the full tables and figures are in the week 15 report.

scope	split	LRU	ARC	S3F	LHD	linear K	tree K	NN K	NN K†	tree FK	NN FK	Belady
btree	test	0.994	0.950	0.862	0.948	0.835	0.834	0.822	0.817	0.829	0.811	0.550
btree	held-out	0.997	0.975	0.908	0.974	0.875	0.867	–	0.851	0.853	0.847	0.615
graph	test	0.990	0.991	1.161	0.865	0.847	0.847	–	0.844	0.806	0.844	0.456
graph	held-out	0.992	0.993	1.256	0.816	0.798	0.765	–	0.760	0.742	0.760	0.482
kv	test	0.984	0.986	1.047	0.906	0.897	0.902	0.850	0.849	0.875	0.843	0.559
kv	held-out	0.982	0.982	1.013	0.901	0.902	0.903	0.845	0.858	0.900	0.854	0.577
lzw	test	0.997	0.968	0.889	0.892	0.821	0.933	–	0.809	0.933	0.803	0.466
lzw	held-out	0.981	0.957	0.866	0.887	0.813	1.068	–	0.841	1.068	0.812	0.409
matmul	held-out	0.879	0.875	1.508	1.036	0.671	0.803	0.868	0.868	0.899	0.868	0.617
sort	test	1.012	1.013	1.321	1.008	0.983	0.879	–	0.891	0.854	0.901	0.830
sort	held-out	0.999	0.999	1.084	1.019	0.933	0.928	–	0.961	0.938	0.924	0.896
global	test	0.992	0.948	0.978	0.926	0.877	0.897	–	–	0.856	0.856	0.591
global	held-out	0.979	0.968	1.020	0.926	0.896	0.894	–	–	0.854	0.847	0.558

Table 10: Faults relative to Clock in the simulator on the original workloads, per scope and split: the classical policies, the linear model over kernel signals (K), trees and nets over the same signals, and the same with oracle information (FK), and Belady (the best possible).

stream	LRU	ARC	S3-FIFO	LHD	Belady	linear K+	linear F+K	linear kv K+
app-bloom	1.000	0.989	0.962	0.998	0.635	0.930	0.926	–
app-chase	0.991	1.002	1.021	0.984	0.683	1.021	1.020	–
app-hashjoin	0.998	1.000	1.012	0.994	0.544	1.138	1.017	–
app-spmv	0.992	1.000	1.026	1.046	0.727	1.239	0.995	–
kv-T	0.983	0.983	1.028	0.916	0.611	–	–	0.980
pat-loop	1.000	1.000	1.000	1.000	0.882	0.894	0.887	–
pat-phase	0.968	0.887	0.899	0.876	0.552	0.899	2.644	–
pat-phaseshort	0.967	0.888	0.909	0.879	0.552	0.918	2.686	–
pat-scanhot	0.992	0.786	0.795	0.866	0.700	0.748	0.744	–
pat-scanhotlo	1.001	0.870	0.874	0.982	0.803	0.829	0.903	–
pat-switch	1.005	0.882	0.877	1.007	0.704	0.817	0.805	–
pat-uniform	1.000	1.000	1.000	1.000	0.658	1.000	1.000	–
pat-zipf060	0.998	0.915	0.881	0.975	0.612	0.845	0.824	–
pat-zipf080	0.994	0.874	0.837	0.932	0.587	0.796	0.773	–
pat-zipf099	0.990	0.846	0.806	0.897	0.565	0.764	0.737	–
pat-zipf120	0.984	0.825	0.783	0.863	0.544	0.737	0.710	–
redis-real	0.929	0.967	1.104	0.973	0.667	1.106	3.000	–
sqlite-real	0.908	0.947	1.082	0.927	0.567	0.934	1.318	–

Table 11: Faults relative to Clock per stream group on workloads no model was trained on (simulator, geometric mean over streams and the three memory sizes).

LeCaR mixes two experts and learns, from the pages each expert evicted too early, which one to trust. As published it does well on btree and kv (0.90 to 0.92, against 0.95 to 0.99 for ARC and LRU), ties them on graph (0.97 to 0.98; LHD is better there), loses on LZW (1.04), and does not reach the linear score on the kernel's signals (0.80 to 0.90 on the same four scopes). In the form a kernel can run it is worse than Clock on most scopes (1.06 to 1.33 on btree, graph, kv and LZW), and the same in the kernel: 1.113 of Clock's faults over 74 streams at 10% memory (interval 1.04 to 1.23), only on the 10% memory size. On the two streams we looked at, the two experts were penalised about equally often, so the weights drift instead of settling on Clock, and the frequency expert, a count that can only be updated at evictions, is a poor expert on its own (the kernel's frequency policy is worse than Clock on average: 1.07 to 1.27). Both versions were checked against independent reference implementations (95 traces, identical faults and evictions). We did

scope	split	LRU	ARC	LHD	LeCaR, as published	LeCaR, kernel form	linear K
btree	test	0.994	0.950	0.948	0.906	1.122	0.835
btree	held-out	0.997	0.975	0.974	0.910	1.065	0.875
graph	test	0.990	0.991	0.865	0.981	1.327	0.847
graph	held-out	0.992	0.993	0.816	0.968	1.321	0.798
kv	test	0.984	0.986	0.906	0.904	1.127	0.897
kv	held-out	0.982	0.982	0.901	0.918	1.149	0.902
lzw	test	0.997	0.968	0.892	1.043	1.056	0.821
lzw	held-out	0.981	0.957	0.887	1.042	1.190	0.813
matmul	held-out	0.879	0.875	1.036	0.851	0.882	0.671
sort	test	1.012	1.013	1.008	0.980	1.016	0.983
sort	held-out	0.999	0.999	1.019	1.012	1.000	0.933
global	held-out	0.979	0.968	0.926	0.958	1.116	0.896

Table 12: A learned-cache baseline from the literature, LeCaR, in the simulator (faults relative to Clock; same cells as the table above). "As published" has experts exact LRU and exact LFU and needs every access, so a kernel cannot run it; "kernel form" uses Clock and the kernel's decayed frequency count as the two experts, reads the accessed bits at evictions, and counts time in evictions. Both use LeCaR's original parameters (learning rate 0.45, history of half the cache per expert).

not run it in the kernel.

C Which pages should a sampled eviction look at?

The kernel runs sample the pages at random. To see whether a cheap rule does better, we replayed the same 74 streams in the simulator at the kernel's memory sizes, with the kernel's global linear score, scanning only the sampled pages (as the kernel does). Faults relative to the full linear score (below 1 is better; workload-weighted geometric mean):

which pages	32 sampled	16 sampled
random (the kernel runs; mean of 3 seeds)	1.002	1.019
the quarter newest loaded, rest random	1.002	1.021
evenly spread over the load order	1.015	1.019
a window in load order that moves on	1.056	1.086
pages not seen at their latest scan first	1.058	1.089
the oldest loaded pages	1.149	1.198

Random is as good as anything we tried, and the three seeds agree to within 0.001. Taking the pages from one end of the load order, or the pages that looked idle, is clearly worse. The even spread ties random at 16 but is worse at 32, because with fewer than 64 resident pages its stride is 1 and it becomes "the 32 oldest pages". Neither cheap rule saves much work: measured exactly in the kernel (68 candidates), random sampling costs 11 305 instructions at 32 and 5 668 at 16, evenly spread 11 275 and 5 681 (the random draws are a negligible part), and the newest-plus-random rule 10 759 and 5 002 (4 to 12% fewer, probably because the newest pages are quickly found to be in use). The simulator and the kernel agree on the same streams: the kernel's sampled score is 0.998 and 1.013 of the full score at 32 and 16, the simulator's 1.002 and 1.019, and kernel faults divided by simulator faults average 1.000 over the 74 streams. An earlier sweep in the simulator found 1.017 and 1.04; it used only 33 streams (those streams alone give 1.016 and 1.039 now), so the difference was the choice of streams, not the kernel.

D How to reproduce

- Kernel runs: `python3 tools/ml2/kernel_eval.py -fracs 0.1 -nonml arck,s3fifo,lhd`; the model runs use `-models cnn-all,...`. Results go to `report/results2/`; the Kaggle packages are in `kaggle_package/round1..round4`.
- Merge: `python3 tools/ml2/merge_kernel_results.py`; intervals: `kernel_ci_classic.py`; tables: `report_w15.py` then `report_w16.py`.
- Kernel policies: `VM_POLICY_ARCK`, `S3FIFO`, `LHD` (5, 6, 7) in `kernel/vmpage.c`; in xv6, `vmrun arck|s3fifo|lhd <program>`.